

--- title: "WB-2026-16 · Deploy Contínuo de Agentes IA — Do Git ao Produção em 5 Minutos" description: "Como criar um pipeline CI/CD completo para agentes IA — com testes, Docker, k8s, feature flags, observabilidade, e rollback automático" tags: [webinar, devops, ci-cd, kubernetes, deploy, feature-flags, observability, ml-ops] nivel: Master → Elite duracao: 90 min data: "2027-01-22" preletor: "Sir. Nexus Alencar (CTO) + convidado Snyk" participacao: "live + replay" pattern: "MMN_IA" --- **WB-2026-16 · Deploy Contínuo de Agentes IA — Do Git ao Produção em 5 Minutos**

Pipeline completo, do `git push` até o agente em produção, com testes automatizados, Docker, Kubernetes, feature flags, observabilidade, e rollback em < 60s. O que o contribuidor paralelo fez, eu mostro como fazer.

****Por Sir. Nexus Alencar · Academ'IA** Nexus Affil'IA'te · 2026 --- #**

Sumário > **• 1. Por que deploy manual é o anti-padrão #1 > **•** 2. Os 4 estágios do deploy moderno > **•** 3. CI/CD com GitHub Actions > **•** 4. Testes: unit, integration, e2e, LLM-as-judge > **•** 5. Docker multi-stage > **•** 6. Kubernetes básico > **•** 7. Feature flags (rollout gradual) > **•** 8. Observabilidade (logs, metrics, traces) > **•** 9. Rollback automático em < 60s > **•** 10. Blue-green e canary release --- **1. Por que deploy manual é o anti-padrão #1****

Em 2024, deploy manual ainda era comum. Em 2026, é **anti-padrão. **Por que:****

	Sem CI/CD	Com CI/CD
30-60 min por deploy	5-10 min	
Erro humano		
Automatizado = consistente		
Difícil de reverter		
Rollback = 1 comando		
Só senior faz		
Qualquer dev		
Deploy 1x/semana		Deploy 10-50x/dia
Lead time: 1 semana		Lead time: 1h

****Adoção em 2026:** 78% das empresas têm CI/CD. Ficar de fora = perder competição. --- **2. Os 4 estágios do deploy moderno****

```

Commit → CI (test) → Build (Docker) → Deploy (k8s) → Monitoring
↓ ↓ ↓ falha falha falha ↓ ↓ ↓ BLOCK bloqueia rollback

```

****Estágio 1 — CI (test)**** - Lint, type check, security scan - Testes unit, integration, e2e - Coverage mínimo (80%)

****Estágio 2 — Build (Docker)**** - Imagem multi-stage (menor) - Tag com git SHA - Push para registry

****Estágio 3 — Deploy (k8s)**** - Rolling update - Health check - Readiness probe

****Estágio 4 — Monitoring**** - Logs, métricas, traces - Alertas - Auto-rollback se métrica ruim

--- ****3. CI/CD com GitHub Actions****

****Pipeline completo**** em 1 arquivo YAML:

```

yaml # .github/workflows/deploy.yml
name: Deploy Agent on: push: branches: [main] jobs: test: runs-on: ubuntu-latest steps: - uses: actions/checkout@v4 - uses: actions/setup-python@v5 with: { python-version: '3.12' } - run: pip install -r requirements.txt -r requirements-dev.txt - run: ruff check . - run: mypy . - run: pytest --cov=app --cov-fail-under=80 build: needs: test if: github.ref == 'refs/heads/main' runs-on: ubuntu-latest steps: - uses: actions/checkout@v4 - name: Build Docker run: | docker build -t agent:${{ github.sha }} . docker tag agent:${{ github.sha }} registry.example.com/agent:${{ github.sha }} docker push registry.example.com/agent:${{ github.sha }} deploy-prod: needs: build environment: production runs-on: ubuntu-latest steps: - name: Deploy run: | echo "${{ secrets.KUBECONFIG }}" | base64 -d > /tmp/kubeconfig KUBECONFIG=/tmp/kubeconfig kubectl set image deployment/agent \ agent=registry.example.com/agent:${{ github.sha }} -n production KUBECONFIG=/tmp/kubeconfig kubectl rollout status deployment/agent -n production

```

****Resultado:**** commit → prod em ****5-10 min****, sem erro humano. --- ****4. Testes: unit, integration, e2e, LLM-as-judge****

****Unit (rápido, isolado):****

```

python def test_skill_consultar_produto(): skill =

```

```

ConsultarProduto() result = skill.execute({"produto_id": "X123"}) assert
result["preco"] > 0 ``` **Integration (com DB/Redis):** ```python def
test_agent_with_real_redis(): agent = MyAgent(redis=real_redis) response =
agent.run("Quanto custa o produto X?") assert "R$" in response ``` **E2E
(sistema completo):** ```python def test_full_user_journey(): session =
start_session(user_id="test") response = send_message(session, "Oi, quanto
custa o curso X?") assert "R$" in response ``` **LLM-as-judge (qualidade do
output):** ```python def test_output_quality(): output = agent.run("Explique
pricing dinâmico") score = judge_llm.score( prompt=f"Output: {output}
\n\nCritérios: relevance, accuracy, helpfulness", criteria="clareza,
completude, utilidade" ) assert score > 0.8 ``` --- **5. Docker multi-stage**
```dockerfile # Estágio 1: build FROM python:3.12-slim AS builder
WORKDIR /app COPY requirements.txt . RUN pip install --user --no-cache-dir
-r requirements.txt # Estágio 2: runtime (imagem final menor) FROM
python:3.12-slim AS runtime WORKDIR /app COPY --from=builder /
root/.local /root/.local ENV PATH=/root/.local/bin:$PATH COPY . . RUN
useradd -m appuser USER appuser HEALTHCHECK --interval=30s CMD
curl -f http://localhost:8000/health || exit 1 EXPOSE 8000 CMD ["uvicorn",
"app.main:app", "--host", "0.0.0.0", "--port", "8000"] ``` **Tamanhos
típicos:** - Sem multi-stage: 1.2GB - Com multi-stage: 280MB - Com
distroless: 80MB --- **6. Kubernetes básico** ```yaml apiVersion: apps/v1
kind: Deployment metadata: name: agent namespace: production spec:
replicas: 3 strategy: type: RollingUpdate rollingUpdate: maxSurge: 1
maxUnavailable: 0 selector: matchLabels: { app: agent } template:
metadata: labels: { app: agent } spec: containers: - name: agent image:
registry.example.com/agent:latest ports: [{ containerPort: 8000 }] env: -
name: OPENAI_API_KEY valueFrom: { secretKeyRef: { name: agent-secrets,
key: openai-key } } resources: requests: { cpu: 200m, memory: 512Mi }
limits: { cpu: 1000m, memory: 2Gi } livenessProbe: httpGet: { path: /health,
port: 8000 } initialDelaySeconds: 30 periodSeconds: 10 readinessProbe:
httpGet: { path: /ready, port: 8000 } initialDelaySeconds: 5 periodSeconds: 5
``` **Comandos úteis:** ```bash # Status kubectl get pods -n production #
Logs kubectl logs -f deployment/agent -n production # Rollback kubectl
rollout undo deployment/agent -n production # Escalar kubectl scale
deployment/agent --replicas=5 -n production ``` --- **7. Feature flags
(rollout gradual)** **Feature flag** = toggle em runtime. ```python from
flagsmith import Flagsmith flagsmith = Flagsmith(environment_key="...")
def new_skill(user_id: str): if flagsmith.is_feature_enabled("skill_v2",
user_id): return skill_v2.execute(...) else: return skill_v1.execute(...) ```
**Estratégias de rollout:** | % usuários | Quando | |-----|-----| | Internal
(só time) | Teste inicial | | Canary (1-5%) | Detectar bugs latentes | | Beta
(10-25%) | Feedback de early adopters | | Gradual (50% → 100%) | Rollout
amplo | **Vantagem:** se der bug, **muda a flag = 0 impacto**. --- **8.
Observabilidade (logs, metrics, traces)** **Logs (Pino):** ```python
logger.info({ "event": "agent.execute", "trace_id": trace_id, "user_id":
user_id, "duration_ms": 245, "cost_cents": 12, "status": "success" }) ```
**Métricas (Prometheus + Grafana):** - Latência p95/p99 - Taxa de erro -
Custo por hora - Volume de requests **Traces (OpenTelemetry + Jaeger):** -
Span: "LLM call" - Span: "DB query" - Trace: jornada completa --- **9.
Rollback automático em < 60s** **Por health check:** - k8s mata pod que
falha 3x health check - Se persiste, rollback automático **Por métrica (auto-
rollback):** ```python def check_and_rollback(): error_rate =

```

```
get_metric("error_rate_5m") p95_latency = get_metric("latency_p95_5m")
cost_per_hour = get_metric("cost_per_hour") if error_rate > 0.10 or
p95_latency > 10 or cost_per_hour > 1000_00: run_kubectrl("rollout undo
deployment/agent -n production") send_alert_to_oncall("Auto-rollback
executed") ``` **Por feature flag:** - Se a nova feature causa problema,
**desliga a flag** (sem redeploy) --- **10. Blue-green e canary release**
**Blue-green:** - 2 ambientes idênticos (blue = atual, green = novo) - Switch
do LB: 1 segundo - Rollback = switch de volta **Canary:** - 5% tráfego na
nova versão - Monitora 30min - Se OK: 25% → 50% → 100% - Se ruim: deleta
canary (rollback) **Quando usar cada:** - **Blue-green:** mudanças
grandes (refactor, upgrade) - **Canary:** mudanças incrementais (nova skill,
prompt) --- *WB-2026-16 · Deploy Contínuo de Agentes IA · Janeiro 2027*
*Por MMN AI-to-AI · 2026 · Licença: CC BY-SA 4.0* *"Deploy manual é o
anti-padrão. Crie um pipeline em 1 semana. Depois é só acumular features
em cima."*
```